

DAT 4기 캡스톤 프로젝트 보고서

부실기업 예측 모형에 관한 연구

DAT 4기

대파

백준홍 이다빈 김세원 신광혁



Data Analysis & Technology



DAT 4기 캡스톤 프로젝트 보고서

부실기업 예측 모형에 관한 연구

A Study on the Predictive Model of Insolvent Companies

이 보고서를 캡스톤 프로젝트 보고서로 제출합니다.

2024년 12월 02일

DAT 4기

대파

백준홍 이다빈 김세원 신광혁

DAT 4기 캡스톤 프로젝트 보고서



Data Analysis & Technology



부실기업 예측 모형에 관한 연구

국문 요약

본 연구는 기업 부실화가 기업과 국가 경제에 미치는 부정적 영향을 고려하여, 딥러닝 모델을 활용한 한계기업 예측의 성능을 분석하고자 하였다. 이를 위해 2020년부터 2024년까지 KOSPI, KOSDAQ, KONEX 및 외부감사 대상기업의 재무비율 데이터를 활용하였으며, 데이터 전처리와 변수 선정 과정을 수행하였다.

분석에는 MLP, CNN, LSTM, GRU 모델을 적용하였으며, 성능 평가는 정확도와 F1-Score를 기준으로 진행하였다. 그 결과, 은닉층 2개(노드 64개)로 구성된 GRU 모델이 가장 우수한 성능을 보였으며, 예측확률을 기준으로 기업을 안전(0.18 미만), 위험(0.18 이상), 매우 위험(0.6 이상)으로 분류하였다.

아울러, 재무비율을 특성에 따라 군집화하고 각 군집별 변수로 부실기업 위험도를 평가하여, 단순한 부실 여부 예측을 넘어 기업 부실에 영향을 미치는 주요 요인을 제시하였다. 본 연구는 딥러닝 기반 예측 모델의 유용성을 입증함과 동시에 기업 부실 예방을 위한 의사결정 지원 가능성을 제안한다.

Abstract

This study examines the predictive performance of deep learning models in identifying marginal firms, considering the negative impacts of corporate insolvency on both businesses and the national economy. Financial ratio data from KOSPI, KOSDAQ, KONEX, and other companies subject to external audits, spanning from 2020 to 2024, were utilized. The data underwent preprocessing and variable selection processes to ensure robustness.

The analysis employed MLP, CNN, LSTM, and GRU models, with performance evaluated based on accuracy and F1-Score. Among these, the GRU model with two hidden layers (64 nodes each) demonstrated superior performance. Using prediction probabilities, firms were classified as safe (below 0.18), at risk (0.18 or above), and highly at risk (0.6 or above).

Additionally, financial ratios were clustered by their characteristics, and variables from each cluster were used to assess the insolvency risk for firms. This approach not only predicts insolvency but also identifies key factors influencing corporate health. The study highlights the utility of deep learning models in predicting marginal firms and suggests their potential for supporting decision-making to prevent corporate insolvency.

DAT 4기

대파

백준홍 이다빈 김세원 신광혁



Data Analysis & Technology



목 차

제 1장 서론	1
제 1절 연구 배경	1
제 2절 연구 목적 및 논문 구성	2
제 2장 데이터	4
제 1절 데이터 소개	4
제 2절 데이터 전처리	4
제 3장 모델링	7
제 1절 연구 모형 설계	7
제 2절 모델 개선 방안	24
제 4장 결론	33
제 1절 연구 결과 분석	33
제 2절 연구한계점 및 제언	37
참고문헌	38



표 목차

[표 1]	5
[표 2]	32

그림 목차

[그림 1]	7
[그림 2]	8
[그림 3]	8
[그림 4]	9
[그림 5]	10
[그림 6]	11
[그림 7]	11
[그림 8]	12
[그림 9]	12
[그림 10]	13
[그림 11]	14
[그림 12]	14
[그림 13]	15
[그림 14]	16
[그림 15]	16
[그림 16]	17



[그림 17]	· · · · ·	19
[그림 18]	· · · · ·	19
[그림 19]	· · · · ·	20
[그림 20]	· · · · ·	20
[그림 21]	· · · · ·	21
[그림 22]	· · · · ·	21
[그림 23]	· · · · ·	22
[그림 24]	· · · · ·	23
[그림 25]	· · · · ·	24
[그림 26]	· · · · ·	25
[그림 27]	· · · · ·	26
[그림 28]	· · · · ·	27
[그림 29]	· · · · ·	27
[그림 30]	· · · · ·	28
[그림 31]	· · · · ·	29
[그림 32]	· · · · ·	30
[그림 33]	· · · · ·	31
[그림 34]	· · · · ·	33
[그림 35]	· · · · ·	34
[그림 36]	· · · · ·	35
[그림 37]	· · · · ·	36



제 1 장 서론

제 1 절 연구 배경

내수 침체가 이어가는 상황에서 수출 회복세마저 둔화되며 기업 실적에도 먹구름이 짙어졌다. 중소·중견 기업은 물론이고 대기업 중에서도 영업 이익으로 이자도 갚지 못하는 잠재적 부실 기업의 수가 점차 증가하고 있다. 기업 데이터 연구소인 CEO 스코어는 올해 11월 27일 국내 500대 기업 중 금융사를 제외한 분기 보고서 제출한 기업 271 곳을 대상으로 지난해 3분기부터 올해 3분기까지 누적 영업이익과 이자 비용을 조사한 보고서를 공개했다. 국내 대기업 중, 영업이익이자보상배율이 1 미만인 기업이 약 20%인 52 곳에 달한다는 게 주요 내용이다. 지난해 같은 기간보다 부실 영업이익이자보상배율 1 미만 기업이 3% 증가했다.¹ 또한 한국경제인협회가 발표한 '2023년 부실기업 추정' 보고서에 따르면, 지난해 금융업계 제외 3만 6425개 사 중 약 11.7%에 해당하는 4천 255개사가 완전자본잠식 상태에 놓인 것으로 추정된 것을 보아, 부실 기업의 수는 꾸준히 상승세였음을 짐작할 수 있다.²

영업이익이자보상배율은 영업 이익을 대출 이자 등 금융 비용으로 나눈 값을 말한다. 이 수치가 1보다 작다는 것은 연간 영업 이익으로 이자도 감당할 수 없다는 것을 의미한다. 즉, 사업 활동을 영위하고 있으나 적정 수준의 이익을 지속적으로 확보하고 있지 못하는 것이다. 이처럼 보유하고 있는 유동성 자산이 기업이 부담하고 있는 채무 중 당장 상환해야 할 부문에 미치지 못하여 채무 불이행의 위험에 놓여 있는 기업을 부실 기업이라고 정의한다. 부실 기업의 증가는 경제 전반에 걸쳐 심각한 영향을 끼치며, 이를 방지할 경우 금융 시스템과 경제 안정성에 중대한 위협이 될 수 있다. 기업의 부실은 기업 자체의 생존 문제로 그치지 않고, 은행권과 금융기관에 신용위험을 전가함으로써 금융 시장의 건전성을 악화시킨다. 특히, 부실기업의 대출이 연체로 이어질 경우 은행의 대출 포트폴리오가 악화되고, 금융권의 부실률이 상승하며, 이는 곧 전반적인 경제 체계에 부정적인 파급 효과를 초래한다.

또한, 이러한 상황이 장기화될 경우 금융기관은 위험 회피를 위한 대출 축소나 금리 인상과 같은 보수적인 정책을 취할 가능성이 높아진다. 이는 중소기업 및 신생 기업의 자금 조달을 어렵게 하고, 경제

¹ "길어지는 경기 침체...늘어난 잠재적 부실기업", 민들레 뉴스, 장박원, 길어지는 경기 침체...늘어난 '잠재적 부실기업' < 경제 > 기사본문 - 세상을 바꾸는 시민언론 민들레

² "한경협 '작년 부실기업 4천개사 넘을듯...최근 5년간 최대'", 한국무역협회 무역 뉴스, n.d., 한경협 "작년 부실기업 4천개사 넘을듯...최근 5년간 최대"



성장률을 저해하며, 고용 시장에까지 악영향을 미친다. 결과적으로 경제 불균형이 심화되고, 사회적 비용이 증가하는 악순환이 발생할 수 있다.

따라서 부실기업을 조기에 식별하고, 이들의 재무 건전성을 평가할 수 있는 효과적인 예측 모델을 구축하는 것은 금융 안정성을 유지하고 경제적 충격을 최소화하기 위한 핵심 과제이다. 본 연구는 부실기업 예측 모델을 개발함으로써, 금융기관 및 정책입안자가 더 나은 의사결정을 내릴 수 있도록 돕고, 장기적으로 경제 체계의 안정성을 강화하는 것을 목표로 한다.

제 2 절 연구 목적

기업 부실화를 예측하기 위하여 과거로부터 많은 연구가 이루어져 왔다. 기업부도 예측은 1960년대부터 본격적으로 연구가 수행되기 시작하였다. 대표적으로 Altman은 5가지 재무변수를 이용하여 다중판별분석(Multiple Discriminant Analysis)를 통해 기업의 부도 위험을 예측하고자 하였다. 그 외에도 로지스틱 회귀모형(Logistic Regression Model)을 활용한 부도예측 연구 등 다양한 통계기법들이 적용된 연구들이 수행되었으며 각 연구는 그 나름의 시사점을 제시하였다.³ 그러나 이러한 기존의 부도기업 예측 연구는 실무적, 방법론적인 한계점을 가진다.

따라서 선행연구의 한계점을 보완하기 위하여, 본 연구에서는 기업 부실화 예측을 위해 부실기업을 예측 대상인 종속변수로 사용하고자 한다. 부실 기업은 개념적으로 경영이 부실하고 재정적 위기에 빠진 기업이라고 정의할 수 있는데, 사업 활동은 영위하고 있으나 기업의 생존을 위한 적정 수준의 이익을 확보하고 있지 못하는 계속 기업을 의미한다. 이러한 부실기업을 그 예측대상으로 활용함으로써 본 연구는 계속기업 간의 성과의 변화를 지속적으로 관찰하여 부실 기업의 위험성이 높은 기업을 판별하고자 한다.

또한, 본 연구에서는 부실 기업을 예측하는 방법으로 딥러닝(Deep Learning) 기법을 사용하고자 한다. 딥러닝은 인공신경망(Artificial Neural Network) 이론에 기초를 둔 것으로 인간의 뉴런(Neuron)과 유사한 구조인 입력층(Input Layer)-은닉층(Hidden Layer)-출력층(Output Layer)의 구조를 가지고 모형을 학습 및 예측하는 기법을 말한다. 딥러닝을 비롯한 머신러닝 기법은 비교적 최근에 회계 및 재무 분야에 도입되기 시작한 방법으로, 이에 속하는 기법들을 활용한 다양한 연구들이 활발하게 이루어지고 있다. 이러한 머신러닝 기법들은 과거로부터 널리 이용되어 온 통계기법들에 비해 더욱 우수한 성과를 보이는 것으로 나타났다. 그러나 딥러닝 기법들을 적용하여 그 예측대상을 부실기업으로 한 연구는 현재 전무한 상황이다. 더욱이 본 연구에서 부실 기업 예측 모형에 category 별로 군집화를 하여, 특정 category 를

³ 권혁건, 이동규&신민수. (2017). RNN(Recurrent Neural Network)를 이용한 기업부도예측모형에서의 회계정보의 동적 변화 연구. 지능정보연구, 23(3), 139-153



분발해야 한다는 솔루션을 제공해주는 연구도 전무하다. 이에 본 연구는 딥러닝 모델을 적용하여 부실 기업 예측을 하고자 한다.

본 연구는 다음과 같이 구성되어 있다. 제 2 장에서는 본 연구에서 활용된 데이터 소개와 데이터에 대한 결측치 처리 및 이상치 처리를 진행한 데이터 전처리 과정에 대해서 소개한다. 제 3 장은 연구 모형을 설계하며 사용된 딥러닝 모형들 분석 결과를 설명하고 최종적으로 분석에 사용된 모델과 이에 대한 해설 기술, 그리고 모델 개선 방안으로 구성되어 있다. 제 4 장은 본 연구의 분석을 요약, 정리하고 연구의 한계점과 향후 연구 방향성을 제시한다.



제 2 장 데이터

제 1 절 데이터 소개

본 연구에서는 부실기업 예측 모형 개발을 위해 2020 년부터 2024 년까지 외부감사 대상 기업 39,861 개의 재무정보를 활용하였다. 데이터는 기업 재무정보 분석 전문 플랫폼인 ValueSearch 를 통해 수집하였다. ValueSearch 는 국내외 기업의 재무정보를 체계적으로 제공하는 플랫폼으로, 재무제표와 산업 동향, 재무비율 분석 등 다양한 정보를 포함하고 있어 기업의 재무 상태 및 성장 가능성을 평가하는 데 중요한 도구로 사용된다. 특히 외부감사 대상 기업 및 공시 데이터를 기반으로 정보를 제공하여 높은 정확성과 신뢰성을 보장하며, 재무 위험 분석, 기업 가치 평가, 투자 판단, 부실 예측 등 다양한 분야에서 활용되고 있다. 본 연구에서도 이러한 ValueSearch 데이터를 활용하여 부실기업 예측 모형 개발에 필요한 기초 자료를 구축하였다.

수집된 데이터는 2020 년부터 2024 년까지의 기간 동안 외부감사 대상 기업의 주요 재무지표를 포함하고 있다. 기업을 분류하기 위해 영업이익이자보상비율(Interest Coverage Ratio, ICR)을 기준으로 사용하였다. 영업이익이자보상비율은 영업활동으로 창출한 이익으로 이자를 지급할 수 있는 능력(이자지급 능력)을 측정하는 재무지표⁴로, 본 연구에서는 이 값이 1 미만일 경우 부실기업으로, 1 이상일 경우 정상기업으로 분류하였다. 영업이익이자보상비율이 1 미만인 기업은 영업이익으로 이자비용조차 충당하지 못하는 상태를 나타내며, 이는 기업의 재무 건전성이 낮아 부실 가능성이 높다는 것을 의미한다. 반대로 정상기업은 영업이익으로 이자비용을 충분히 감당할 수 있는 상태를 의미한다.

본 데이터는 부실기업 예측 모형 개발에 있어 신뢰성과 타당성을 확보하기 위한 기초 자료로 활용되며, 이후 데이터 전처리를 통해 분석에 적합한 형태로 가공될 예정이다.

제 2 절 데이터 전처리

데이터 전처리 과정은 결측값 처리, 이상치 처리, 변수 선택의 세 단계로 이루어졌다. 2020 년부터 2024 년까지의 영업이익이자보상비율(Interest Coverage Ratio, ICR)을 계산한 후, 결측값을 처리하였다. 결측값 처리는 다음과 같은 단계를 거쳐 진행되었다.

⁴ 한국금융연구원



2.2.1. 결측값 처리

1. 결측치가 높은 데이터 제거:
 - 2024 년 데이터의 경우, 결측값의 비율이 높아 분석의 정확성에 부정적인 영향을 미칠 가능성이 크다고 판단되어 해당 데이터를 제거하였다.
2. 행 제거:
 - 영업이익이자보상비율의 값이 결측인 경우, 분석에 활용할 수 없으므로 해당 행을 삭제하였다.
3. 중앙값 대체:
 - 결측값이 존재하는 경우, 각 변수의 중앙값(median)으로 대체하였다.
 - 이는 0 값 대체 시 비율값의 의미 왜곡 가능성과, 평균값 대체 시 극단값(outlier)의 영향을 고려한 선택이다.
4. 금융기업 데이터 제외:
 - 금융 기업의 경우, ValueSearch 데이터 내에서 일부 정보가 비공개되어 있어 충분한 분석이 어려웠으므로 해당 데이터를 제외하였다.

변수	결측치 개수	결측치 비율(%)
2020/영업이익이자보상비율	8768	22.6921
2021/영업이익이자보상비율	6378	16.5066
2022/영업이익이자보상비율	4472	11.5738
2023/영업이익이자보상비율	4810	12.4486
2024/영업이익이자보상비율	37943	98.1987

[표 1. 결측치 개수 및 비율]

2.2.2. 이상치 처리

이상치(outlier) 처리는 데이터의 분포를 안정화하고 분석의 신뢰성을 높이기 위해 윈저라이징(Winsorizing) 기법을 활용하였다.

윈저라이징은 극단값을 상한 및 하한 값으로 대체하여 이상치가 분석 결과에 미치는 영향을 줄이는 방법으로, 본 연구에서는 데이터의 상위 및 하위 1%의 값을 적절히 조정하여 이상치를 처리하였다.

2.2.3. 변수 선택

변수 선택 과정은 t 검정과 로지스틱 회귀분석을 통해 수행되었으며, 이를 통해 부실기업 예측에 불필요한 변수를 제거하였다.

t 검정은 두 집단 간 평균의 차이를 검정하는 통계적 방법으로, 본 연구에서는 부실기업과 비부실기업 간 특정 변수의 평균 차이가 유의미한 지를 검정하였다. 검정의 귀무가설(H_0)은 "부실기업 그룹과 비부실기업 그룹 간에 특정 변수의 평균 차이가 없다"는 가설로 설정되었으며, 이를 검증한 결과 평균 차이가 유의미하지 않은 변수 12 개를 제거하였다.

이후 로지스틱 회귀분석을 활용하여 독립변수가 종속변수(부실 여부)와 유의미한 관계를 갖는지를 확인하였다. 로지스틱 회귀분석은 이항형 종속변수를 예측할 때 사용되는 통계적 기법으로, 변수 간의 관계를 분석하여 중요하지 않은 변수들을 제거하는 데 유용하다. 본 연구에서는 후진제거법(Backward Elimination)을 사용하여 불필요한 변수를 점진적으로 제거하였다. 후진제거법은 전체 변수를 포함한 상태에서 시작하여, 유의미하지 않은 변수부터 하나씩 제거하면서 최적의 변수 집합을 선택하는 방법이다. 이 과정을 통해 추가적으로 120 개의 변수를 제거하였다.

이와 같은 전처리 과정을 거친 최종 데이터셋은 총 27,603 개의 기업으로 구성되었다. 정상기업은 22,469 개, 부실기업은 5,134 개로 나뉘었으며, 이는 초기 데이터셋(39,861 개)에서 불필요한 데이터와 변수를 제거하여 분석에 적합한 형태로 가공된 결과이다. 본 연구는 이렇게 전처리 된 데이터를 기반으로 부실기업 예측 모델을 개발하였다.



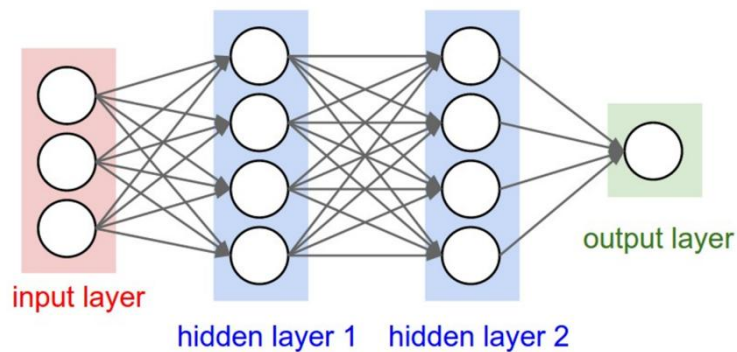
제 3 장 모델링

제 1 절 연구 모형 설계

3.1.1. 모형 개요

본 연구에서는 부실기업 예측을 위해 MLP, CNN, LSTM, GRU 의 네 가지 딥러닝 모형을 선정하여 모형 간의 비교를 통해 최적의 모형을 선정하고자 하였다. 각 모형의 기본적인 구조는 다음과 같다.

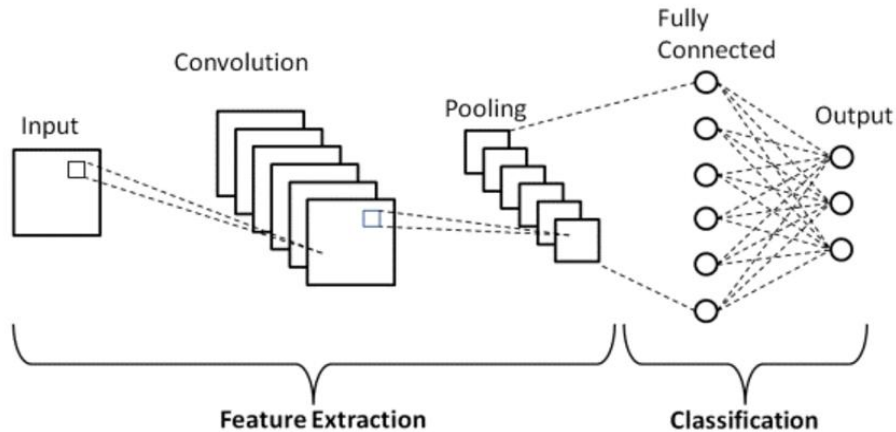
- MLP (다층 퍼셉트론): MLP는 입력층, 여러 개의 은닉층, 출력층으로 구성된 인공신경망 구조이다. 각 층은 비선형 활성화 함수를 사용하여 입력 데이터의 복잡한 패턴을 학습할 수 있다. MLP 는 주로 정형 데이터에서 효과적이며, 예측 및 분류 문제에 널리 사용된다.



[그림 1. MLP 모형 구조⁵⁾]

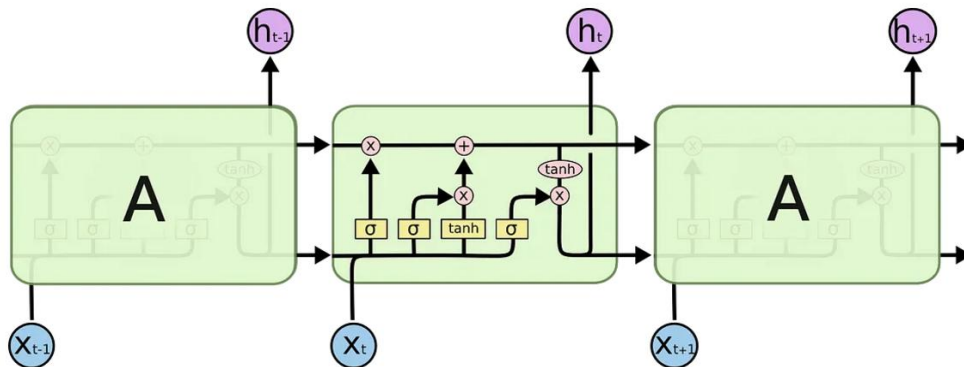
- CNN (합성곱 신경망): CNN은 이미지 처리에 주로 사용되지만, 데이터의 공간적 패턴을 학습하는데 매우 효과적이다. CNN 은 합성곱 층과 풀링 층을 통해 특징을 추출하고, 최종적으로 완전 연결층을 통해 예측을 수행한다. 비정형 데이터에서도 CNN 의 패턴 인식 능력을 활용할 수 있어, 비즈니스 데이터 분석에도 응용 가능하다.

⁵ <https://buomsoo-kim.github.io/keras/2018/04/21/Easy-deep-learning-with-Keras-1.md/>



[그림 2. CNN 모형 구조⁶⁾

- LSTM(장기 단기 메모리): 순환 신경망(RNN)의 한 종류로, 시계열 데이터의 장기 의존성을 효과적으로 처리하기 위해 설계되었으며, 입력 게이트, 망각 게이트, 출력 게이트를 통해 정보를 관리한다. 각 게이트는 필요한 정보를 선택적으로 저장하거나 망각하여 정보가 흐를 수 있는 Cell State 를 생성하는데, 이 구조는 LSTM 이 복잡한 의존관계를 갖는 데이터를 학습하게 해준다. LSTM 은 비선형 활성화 함수를 이용해 다양한 패턴을 모델링하고 시계열 데이터를 다룰 때 특히 유용하며, 금융, 자연어 처리, 음성 인식 등 다양한 분야에서 폭넓게 활용되고 있다.

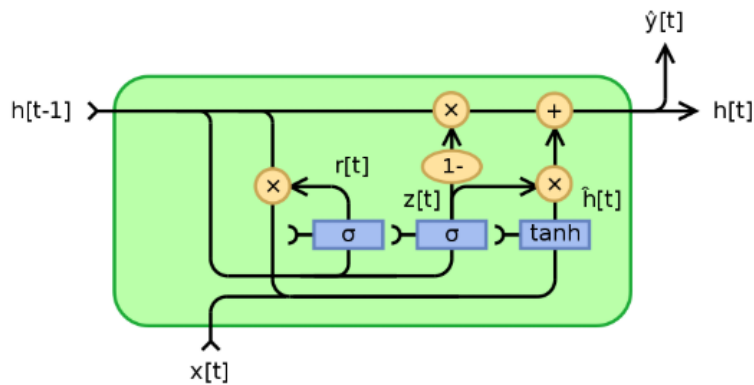


[그림 3. LSTM 모형 구조⁷⁾

⁶⁾ <https://www.upgrad.com/blog/basic-cnn-architecture/>

⁷⁾ <https://www.upgrad.com/blog/basic-cnn-architecture/>

- GRU(게이트 순환 유닛): LSTM 의 경량화 된 버전으로, 두 가지 주요 게이트인 업데이트 게이트와 리셋 게이트로 구성되어 있다. 이 구조는 GRU 가 LSTM 보다 훨씬 간단하게 설계되어 있어 더 빠른 학습과 적은 메모리 사용을 가능하게 한다. GRU 는 시퀀스 데이터가 짧고 처리 속도가 중요한 경우에 매우 효과적이며, 자연어 처리, 음성 인식, 주식 가격 예측 등 다양한 분야에서 강력한 성능을 발휘한다. 특히, GRU 는 과거 정보를 얼마나 잊고 새로운 정보를 얼마나 반영해야 할지를 조절하여, 데이터의 연속성과 시간적 패턴을 효과적으로 학습하는데 탁월한 장점을 가지고 있다.



[그림 4. GRU 모형 구조⁸⁾

⁸⁾ <https://paperswithcode.com/method/gru>

3.1.2. 모형 설계

부실기업 예측을 위한 딥러닝 모형의 설계는 다음과 같은 주요 단계로 구성하였다. 각 모형의 특성에 따라 데이터의 형태를 변환해주는 과정을 통해 모형이 효과적으로 데이터를 처리하고 학습할 수 있도록 하였다.

1. 데이터 준비 및 전처리

본 연구에서는 2021 년과 2022 년의 재무비율 데이터와 타겟변수인 2023/부실기업 변수를 이용하여 모형을 학습하고자 하였다.

```
# 종속 변수 설정
y = data_21to22['2023/부실기업']

# 연도별 특성 분리
X_2021 = data_21to22.filter(like='2021') # 2021년 데이터
X_2022 = data_21to22.filter(like='2022') # 2022년 데이터

# 재무비율 추출 함수
def extract_ratio_columns(df, year):
    return df.columns.str.replace(f'{year}/', '')

# 2021년과 2022년의 재무비율 추출
ratios_2021 = extract_ratio_columns(X_2021, '2021')
ratios_2022 = extract_ratio_columns(X_2022, '2022')
```

[그림 5. 2021년, 2022년도 변수 및 타겟변수 추출]

데이터 형태에서 LSTM, GRU 와 같은 시계열 모형의 경우 학습할 때 데이터의 시계열 특성을 반영하여 학습할 수 있도록 timesteps 를 지정해주어야 한다. 구축된 데이터셋의 변수는 연도별 동일 종류로 선택된 형태가 아니므로 형태를 변환해주어야 한다.

본 연구에서는 시계열 모형을 학습하는 데이터셋의 경우 연도별 동일 변수들로 채워진 데이터셋으로 구축하여 시계열 모형을 학습할 때는 해당 데이터셋으로 학습하도록 데이터 셋을 구분하였다. 이때

재무비율을 21 년도, 22 년도 데이터를 활용하는 점에 있어 timesteps 을 2 로 지정하여 데이터셋을 구축하였다.

```
# 공통 재무비율 찾기
common_ratios = ratios_2021.intersection(ratios_2022)

# 공통 재무비율을 포함하는 변수명 생성
common_columns_2021 = [f'2021/{ratio}' for ratio in common_ratios]
common_columns_2022 = [f'2022/{ratio}' for ratio in common_ratios]

# 공통 변수만 사용하여 데이터 결합
X_2021_common = X_2021[common_columns_2021]
X_2022_common = X_2022[common_columns_2022]

# 전체 특성 결합 (타임 스텝을 위해 배열로 변환)
X_common = pd.concat([X_2021_common, X_2022_common], axis=1)

# 모든 변수를 사용하여 데이터 결합
X_all = pd.concat([X_2021, X_2022], axis=1)
```

[그림 6. 연도별 동일 변수 추출]

```
# 데이터 분할 (70:30 비율)
X_common_train, X_common_test, y_train, y_test = train_test_split(X_common, y, test_size=0.3, random_state=RANDOM_STATE)
X_all_train, X_all_test, _, _ = train_test_split(X_all, y, test_size=0.3, random_state=RANDOM_STATE)

# 데이터 정규화
scaler_common = StandardScaler()
X_common_train_scaled = scaler_common.fit_transform(X_common_train)
X_common_test_scaled = scaler_common.transform(X_common_test)

scaler_all = StandardScaler()
X_all_train_scaled = scaler_all.fit_transform(X_all_train)
X_all_test_scaled = scaler_all.transform(X_all_test)

# 변수명을 유지하기 위해 DataFrame으로 변환
X_common_train_scaled_df = pd.DataFrame(X_common_train_scaled, columns=X_common_train.columns)
X_common_test_scaled_df = pd.DataFrame(X_common_test_scaled, columns=X_common_test.columns)

X_all_train_scaled_df = pd.DataFrame(X_all_train_scaled, columns=X_all_train.columns)
X_all_test_scaled_df = pd.DataFrame(X_all_test_scaled, columns=X_all_test.columns)

# 타임 스텝 설정
timesteps = 2 # 2021년과 2022년으로 2개의 타임 스텝으로 설정 # (샘플 수, 타임 스텝, 특성 수)
X_train_lstm = X_common_train_scaled_df.values.reshape((X_common_train_scaled_df.shape[0], timesteps, -1))
X_test_lstm = X_common_test_scaled_df.values.reshape((X_common_test_scaled_df.shape[0], timesteps, -1))
```

[그림 7. 데이터 전처리 및 시계열 모델 학습을 위한 모델 구축]

```

# 타임 스텝 확인
print("X_train_lstm shape:", X_train_lstm.shape)
print("X_test_lstm shape:", X_test_lstm.shape)

# 첫 번째 샘플의 타임 스텝별 데이터 확인
print("첫 번째 샘플의 첫 번째 타임 스텝 데이터:", X_train_lstm[0, 0, :])
print("첫 번째 샘플의 두 번째 타임 스텝 데이터:", X_train_lstm[0, 1, :])

X_train_lstm shape: (19322, 2, 21)
X_test_lstm shape: (8281, 2, 21)
첫 번째 샘플의 첫 번째 타임 스텝 데이터: [ 0.03807207 -0.0020328  0.05400356  0.05899398  0.07017119  0.14827777
 0.00180214  0.61243826 -0.33427654 -0.185362  -0.19290163 -0.19358192
-0.03935053 -0.35014204  0.41301943  0.25458916  0.33200608 -0.04324597
-0.03086146 -0.12714346 -0.22720773]
첫 번째 샘플의 두 번째 타임 스텝 데이터: [-0.54167093 -0.62132018 -0.51073733 -0.55508638 -0.57985577  0.097176
-0.0018005  -0.13981697 -0.35060469 -0.18735672 -0.19095395 -0.19360187
-0.06447123 -0.34301056  0.35340432  0.24664365  0.25629906 -0.07094113
-0.02741867 -0.12251024 -0.21648227]

```

[그림 8. timesteps를 지정한 데이터셋 형태]

```

# 데이터 shape 출력
print("X_all_train_scaled_df shape:", X_all_train_scaled_df.shape)
print("X_all_test_scaled_df shape:", X_all_test_scaled_df.shape)
print("X_train_lstm shape:", X_train_lstm.shape)
print("X_test_lstm shape:", X_test_lstm.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# MLP와 CNN 학습에 사용
# MLP와 CNN 테스트에 사용
# LSTM과 GRU 학습에 사용
# LSTM과 GRU 테스트에 사용
# 종속 변수 학습 데이터
# 종속 변수 테스트 데이터

X_all_train_scaled_df shape: (19322, 86)
X_all_test_scaled_df shape: (8281, 86)
X_train_lstm shape: (19322, 2, 21)
X_test_lstm shape: (8281, 2, 21)
y_train shape: (19322,)
y_test shape: (8281,)

```

[그림 9. timesteps를 지정한 데이터셋 형태]

2. 모델 아키텍처

딥러닝 모델 중에서 MLP, CNN, LSTM, GRU 모델을 비교하여 최적의 모델을 선정하고자 하였다. 이때 은닉층의 수를 2로 고정하고 노드 수를 64, 128, 256인 3가지로 설정하여 모델의 아키텍처를 구축하였다.

```
#----- 모델 구축 (64) -----#

# MLP 모델
mlp_model_64 = Sequential()
mlp_model_64.add(Dense(64, activation='relu', input_shape=(X_all_train_scaled.shape[1],)))
mlp_model_64.add(Dense(64, activation='relu'))
mlp_model_64.add(Dense(1, activation='sigmoid'))
mlp_model_64.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# CNN 모델
cnn_model_64 = Sequential()
cnn_model_64.add(Reshape((X_all_train_scaled.shape[1], 1), input_shape=(X_all_train_scaled.shape[1],)))
cnn_model_64.add(Conv1D(64, kernel_size=2, activation='relu', padding='same'))
cnn_model_64.add(Conv1D(64, kernel_size=2, activation='relu', padding='same'))
cnn_model_64.add(Flatten())
cnn_model_64.add(Dense(1, activation='sigmoid'))
cnn_model_64.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# LSTM 모델
lstm_model_64 = Sequential()
lstm_model_64.add(LSTM(64, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
lstm_model_64.add(LSTM(64, activation='relu'))
lstm_model_64.add(Dense(1, activation='sigmoid'))
lstm_model_64.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# GRU 모델
gru_model_64 = Sequential()
gru_model_64.add(GRU(64, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
gru_model_64.add(GRU(64, activation='relu'))
gru_model_64.add(Dense(1, activation='sigmoid'))
gru_model_64.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])
```

[그림 10. 노드 수 64인 딥러닝 모델의 아키텍처]

```
#----- 모델 구축 (128) -----#

# MLP 모델
mlp_model_128 = Sequential()
mlp_model_128.add(Dense(128, activation='relu', input_shape=(X_all_train_scaled_df.shape[1],)))
mlp_model_128.add(Dense(128, activation='relu'))
mlp_model_128.add(Dense(1, activation='sigmoid'))
mlp_model_128.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# CNN 모델
cnn_model_128 = Sequential()
cnn_model_128.add(Reshape((X_all_train_scaled_df.shape[1], 1), input_shape=(X_all_train_scaled_df.shape[1],)))
cnn_model_128.add(Conv1D(128, kernel_size=2, activation='relu', padding='same'))
cnn_model_128.add(Conv1D(128, kernel_size=2, activation='relu', padding='same'))
cnn_model_128.add(Flatten())
cnn_model_128.add(Dense(1, activation='sigmoid'))
cnn_model_128.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# LSTM 모델
lstm_model_128 = Sequential()
lstm_model_128.add(LSTM(128, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
lstm_model_128.add(LSTM(128, activation='relu'))
lstm_model_128.add(Dense(1, activation='sigmoid'))
lstm_model_128.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# GRU 모델
gru_model_128 = Sequential()
gru_model_128.add(GRU(128, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
gru_model_128.add(GRU(128, activation='relu'))
gru_model_128.add(Dense(1, activation='sigmoid'))
gru_model_128.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])
```

[그림 11. 노드 수 128인 딥러닝 모델의 아키텍처]

```
#----- 모델 구축 (256) -----#

# MLP 모델
mlp_model_256 = Sequential()
mlp_model_256.add(Dense(256, activation='relu', input_shape=(X_all_train_scaled_df.shape[1],)))
mlp_model_256.add(Dense(256, activation='relu'))
mlp_model_256.add(Dense(1, activation='sigmoid'))
mlp_model_256.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# CNN 모델
cnn_model_256 = Sequential()
cnn_model_256.add(Reshape((X_all_train_scaled_df.shape[1], 1), input_shape=(X_all_train_scaled_df.shape[1],)))
cnn_model_256.add(Conv1D(256, kernel_size=2, activation='relu', padding='same'))
cnn_model_256.add(Conv1D(256, kernel_size=2, activation='relu', padding='same'))
cnn_model_256.add(Flatten())
cnn_model_256.add(Dense(1, activation='sigmoid'))
cnn_model_256.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# LSTM 모델
lstm_model_256 = Sequential()
lstm_model_256.add(LSTM(256, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
lstm_model_256.add(LSTM(256, activation='relu'))
lstm_model_256.add(Dense(1, activation='sigmoid'))
lstm_model_256.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# GRU 모델
gru_model_256 = Sequential()
gru_model_256.add(GRU(256, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
gru_model_256.add(GRU(256, activation='relu'))
gru_model_256.add(Dense(1, activation='sigmoid'))
gru_model_256.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])
```

[그림 12. 노드 수 256인 딥러닝 모델의 아키텍처]

3. 모델 학습

구축한 모델의 아키텍처를 통해 은닉층 수 2 개인 동일 조건에서 노드 수 64, 128, 256 에 따라 MLP, CNN, LSTM, GRU 모델을 학습하였다. 이를 통해 각 노드별로 모델의 성능을 비교하여 최적의 모델을 선정하고자 하였다.

```
#----- 모델 학습 -----#

# MLP 학습 및 평가
mlp_history_64 = mlp_model_64.fit(X_all_train_scaled, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
mlp_predictions_proba_64 = mlp_model_64.predict(X_all_test_scaled)
mlp_predictions_64 = (mlp_predictions_proba_64 > 0.5).astype(int)

# CNN 학습 및 평가
X_train_cnn = X_all_train_scaled.reshape((X_all_train_scaled.shape[0], X_all_train_scaled.shape[1], 1))
X_test_cnn = X_all_test_scaled.reshape((X_all_test_scaled.shape[0], X_all_test_scaled.shape[1], 1))
cnn_history_64 = cnn_model_64.fit(X_train_cnn, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
cnn_predictions_proba_64 = cnn_model_64.predict(X_test_cnn)
cnn_predictions_64 = (cnn_predictions_proba_64 > 0.5).astype(int)

# LSTM 학습 및 평가
lstm_history_64 = lstm_model_64.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
lstm_predictions_proba_64 = lstm_model_64.predict(X_test_lstm)
lstm_predictions_64 = (lstm_predictions_proba_64 > 0.5).astype(int)

# GRU 학습 및 평가
gru_history_64 = gru_model_64.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
gru_predictions_proba_64 = gru_model_64.predict(X_test_lstm)
gru_predictions_64 = (gru_predictions_proba_64 > 0.5).astype(int)
```

[그림 13. 노드 수 64인 딥러닝 모델의 학습]

```
#----- 모델 학습 -----#

# MLP 학습 및 평가
mlp_history_128 = mlp_model_128.fit(X_all_train_scaled_df, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
mlp_predictions_proba_128 = mlp_model_128.predict(X_all_test_scaled_df)
mlp_predictions_128 = (mlp_predictions_proba_128 > 0.5).astype(int)

# CNN 학습 및 평가
X_train_cnn = X_all_train_scaled_df.values.reshape((X_all_train_scaled_df.shape[0], X_all_train_scaled_df.shape[1], 1))
X_test_cnn = X_all_test_scaled_df.values.reshape((X_all_test_scaled_df.shape[0], X_all_test_scaled_df.shape[1], 1))
cnn_history_128 = cnn_model_128.fit(X_train_cnn, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
cnn_predictions_proba_128 = cnn_model_128.predict(X_test_cnn)
cnn_predictions_128 = (cnn_predictions_proba_128 > 0.5).astype(int)

# LSTM 학습 및 평가
lstm_history_128 = lstm_model_128.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
lstm_predictions_proba_128 = lstm_model_128.predict(X_test_lstm)
lstm_predictions_128 = (lstm_predictions_proba_128 > 0.5).astype(int)

# GRU 학습 및 평가
gru_history_128 = gru_model_128.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
gru_predictions_proba_128 = gru_model_128.predict(X_test_lstm)
gru_predictions_128 = (gru_predictions_proba_128 > 0.5).astype(int)
```

[그림 14. 노드 수 128인 딥러닝 모델의 학습]

```
#----- 모델 학습 -----#

# MLP 학습 및 평가
mlp_history_256 = mlp_model_256.fit(X_all_train_scaled_df, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
mlp_predictions_proba_256 = mlp_model_256.predict(X_all_test_scaled_df)
mlp_predictions_256 = (mlp_predictions_proba_256 > 0.5).astype(int)

# CNN 학습 및 평가
X_train_cnn = X_all_train_scaled_df.values.reshape((X_all_train_scaled_df.shape[0], X_all_train_scaled_df.shape[1], 1))
X_test_cnn = X_all_test_scaled_df.values.reshape((X_all_test_scaled_df.shape[0], X_all_test_scaled_df.shape[1], 1))
cnn_history_256 = cnn_model_256.fit(X_train_cnn, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
cnn_predictions_proba_256 = cnn_model_256.predict(X_test_cnn)
cnn_predictions_256 = (cnn_predictions_proba_256 > 0.5).astype(int)

# LSTM 학습 및 평가
lstm_history_256 = lstm_model_256.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
lstm_predictions_proba_256 = lstm_model_256.predict(X_test_lstm)
lstm_predictions_256 = (lstm_predictions_proba_256 > 0.5).astype(int)

# GRU 학습 및 평가
gru_history_256 = gru_model_256.fit(X_train_lstm, y_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)
gru_predictions_proba_256 = gru_model_256.predict(X_test_lstm)
gru_predictions_256 = (gru_predictions_proba_256 > 0.5).astype(int)
```

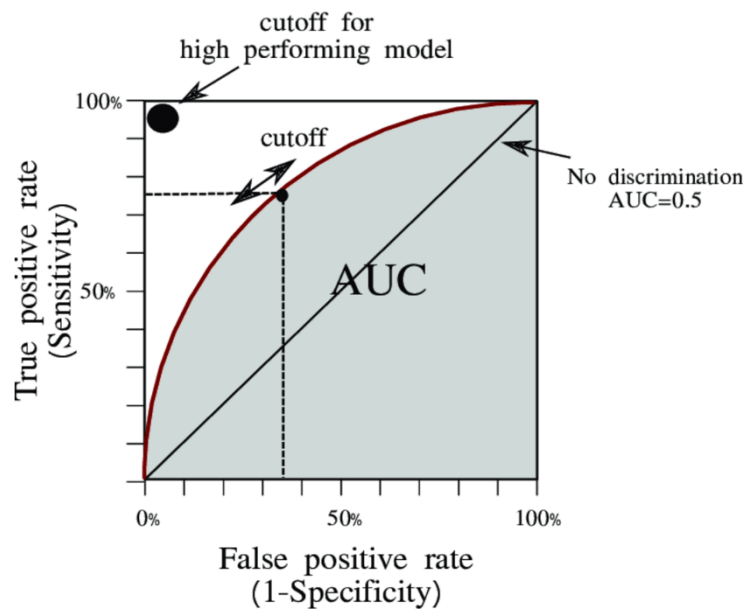
[그림 15. 노드 수 256인 딥러닝 모델의 학습]

3.1.3. 모형간 성능 비교

1. 평가 지표

모형의 성능을 평가하기 위해 주요 지표 중에서는 ROC 곡선, 정확도(Accuracy), 그리고 F1-score 를 사용하였다.

ROC 곡선은 Receiver Operating Characteristic Curve 의 줄임말로, 모형이 긍정 클래스와 부정 클래스를 얼마나 잘 구분하는지를 시각적으로 나타낸다. X 축이 False Positive Rate(FPR), Y 축이 True Positive Rate(TPR)를 나타내며, 다양한 분류 임계값을 사용하여 모형의 성능을 평가할 수 있다. ROC 곡선 아래 면적, 즉 AUC(Area Under Curve)는 모형의 전반적인 성능을 나타내는 좋은 척도로, AUC 값이 1 에 가까울수록 모형이 더 뛰어난 성능을 보인다고 할 수 있다.



[그림 16. Roc 곡선]

정확도(Accuracy)는 전체 샘플 중에서 모델이 올바르게 예측한 비율을 측정한다. 정확도는 다음과 같은 공식을 통해 계산된다.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

여기서 TP 는 True Positives, TN 은 True Negatives, FP 는 False Positives, FN 은 False Negatives 를 의미한다. 그렇지만, 정확도는 클래스의 불균형이 심한 데이터셋에서는 한계가 있을 수 있다. 예를 들어, 불균형 데이터셋에서 많은 경우를 부정 클래스라고 예측할 경우 높은 정확도를 기록할 수 있지만, 실제로는 긍정 클래스를 잘 예측하지 못할 수 있다.

F1-score 는 Precision 과 Recall 의 조화 평균으로, 이 두 평가지표를 모두 고려하여 모델의 성능을 보다 정확히 평가할 수 있다. F1-score 는 다음과 같은 수식으로 계산된다.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

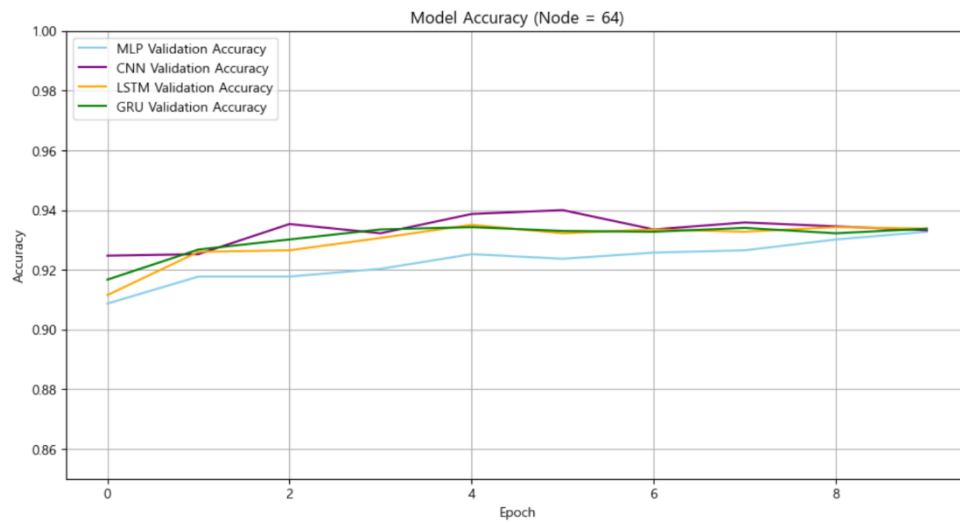
이때 Precision 은 모델이 긍정 불량으로 예측한 것 중 실제로도 긍정인 비율을 나타내고, Recall 은 실제 긍정 중 모델이 올바르게 긍정으로 예측한 비율이다. F1-score 는 데이터셋의 클래스 불균형이 있는 경우 특히 유용하며, Precision 와 Recall 값의 균형을 맞추으로써 실제 모델의 성능을 보다 신뢰성 있게 반영한다.

2. 성능 비교

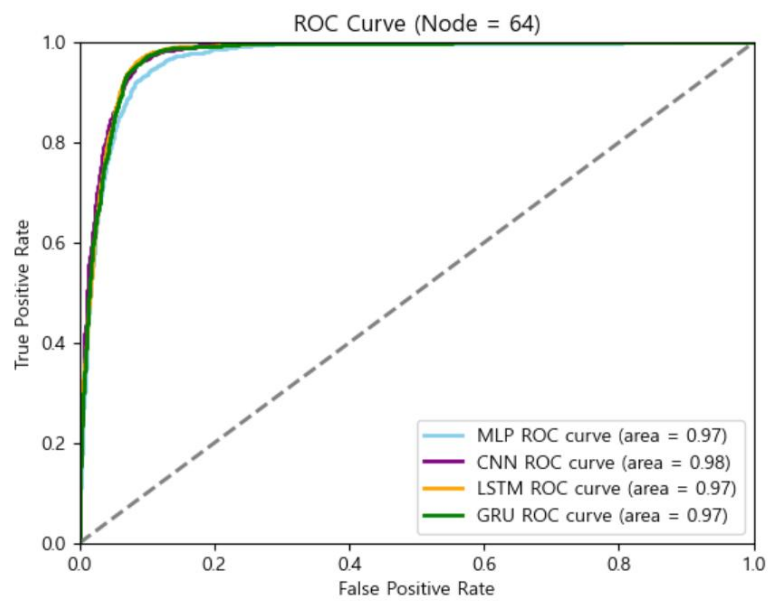
본 연구에서는 데이터 불균형을 고려하여 우선적으로 F1-score 를 기준으로 판단하였고 추가적으로 ROC 곡선, 정확도(Accuracy)를 비교하며 최적의 모델을 선정하였다.



- 노드 수 64

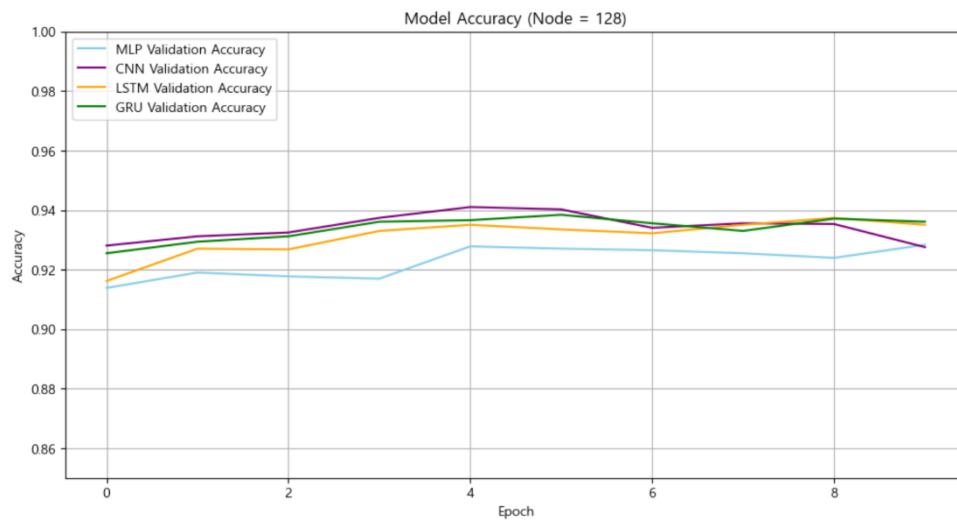


[그림 17. 노드 수 64 일 때 Accuracy]

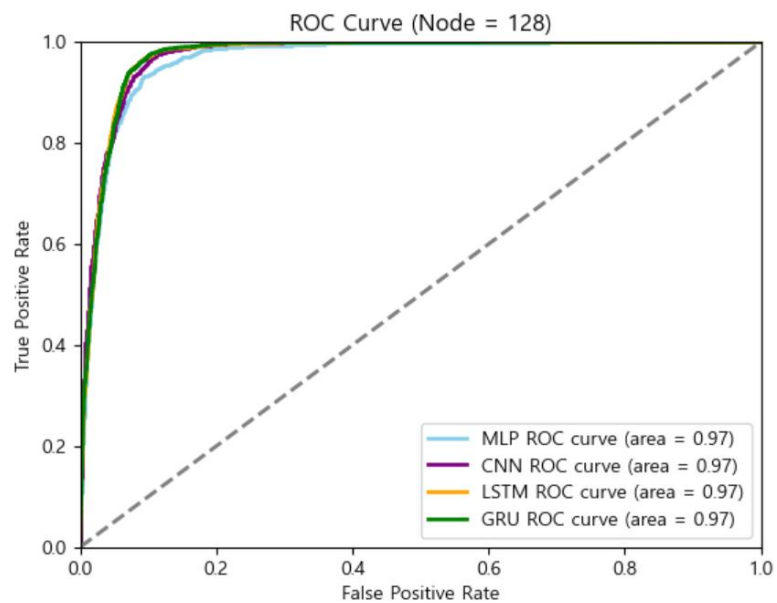


[그림 18. 노드 수 64 일 때 Roc 곡선 및 AUROC]

- 노드 수 128

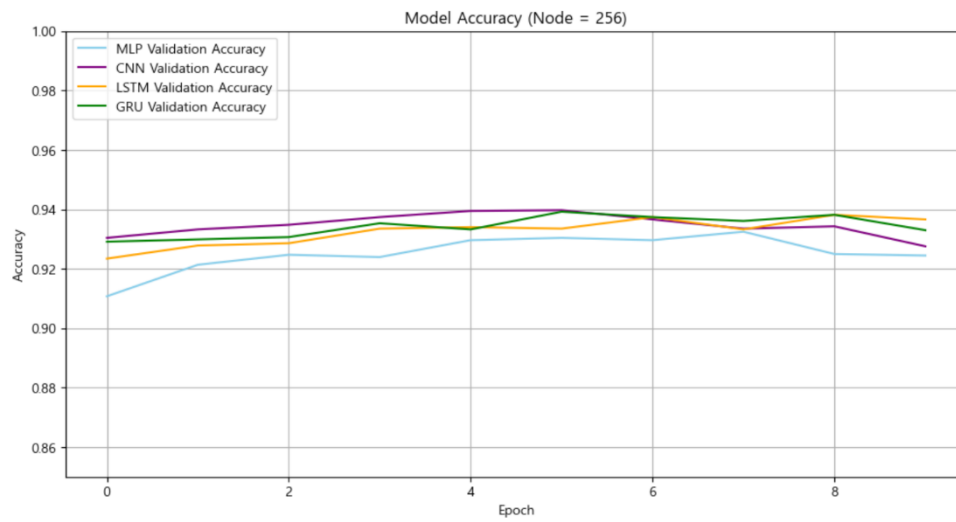


[그림 19. 노드 수 128 일 때 Accuracy]

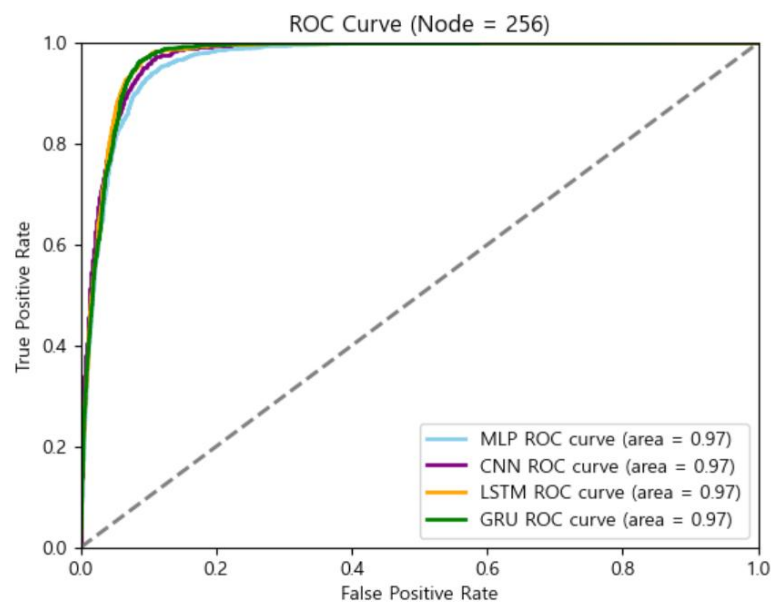


[그림 20. 노드 수 128 일 때 Roc 곡선 및 AUROC]

- 노드 수 256

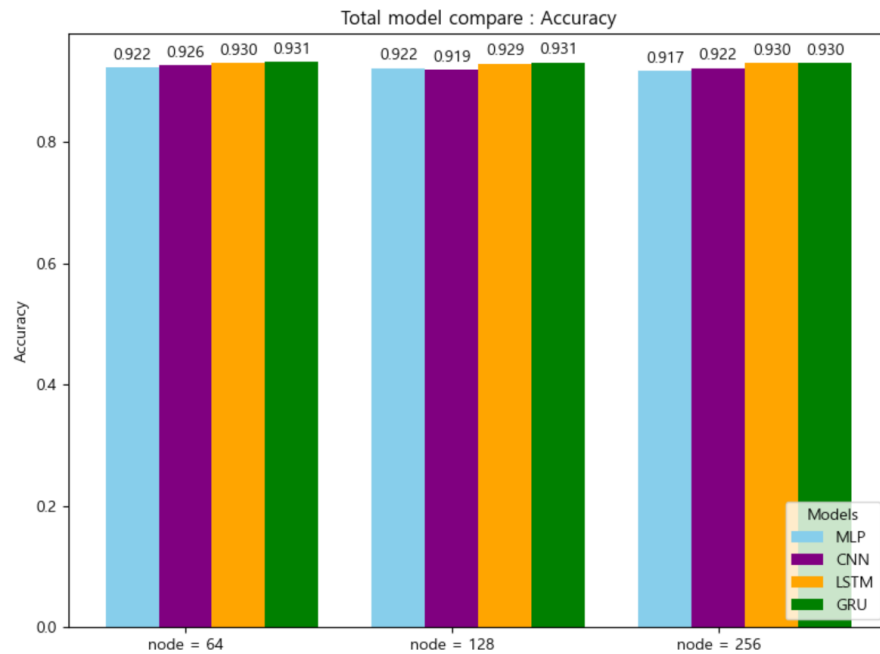


[그림 21. 노드 수 256 일 때 Accuracy]

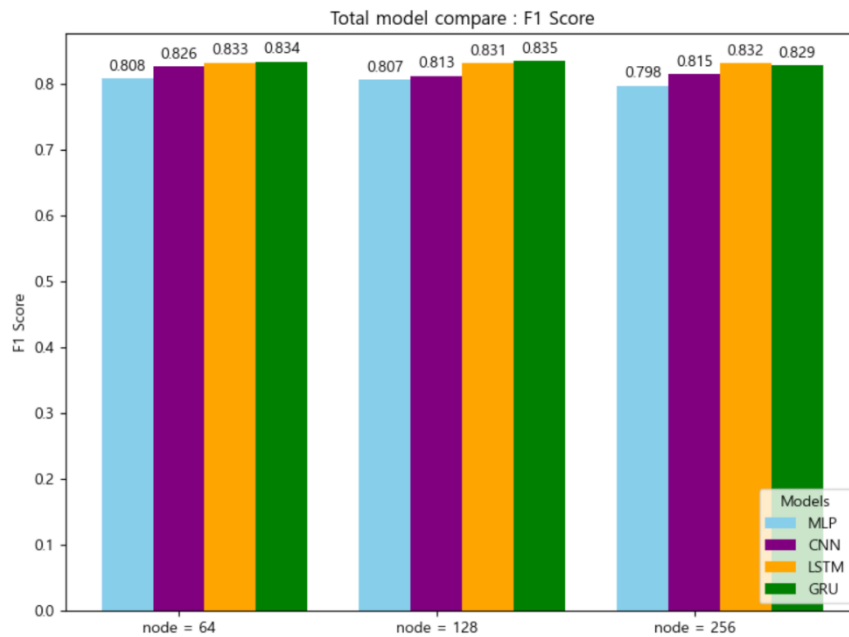


[그림 22. 노드 수 256 일 때 Roc 곡선 및 AUROC]

모델별로 노드 수에 따라 accuracy 및 f1-score 를 시각화 하면 다음과 같다.



[그림 23. 모델별로 노드 수에 따라 Accuracy]



[그림 24. 모델별로 노드 수에 따라 F1-score]

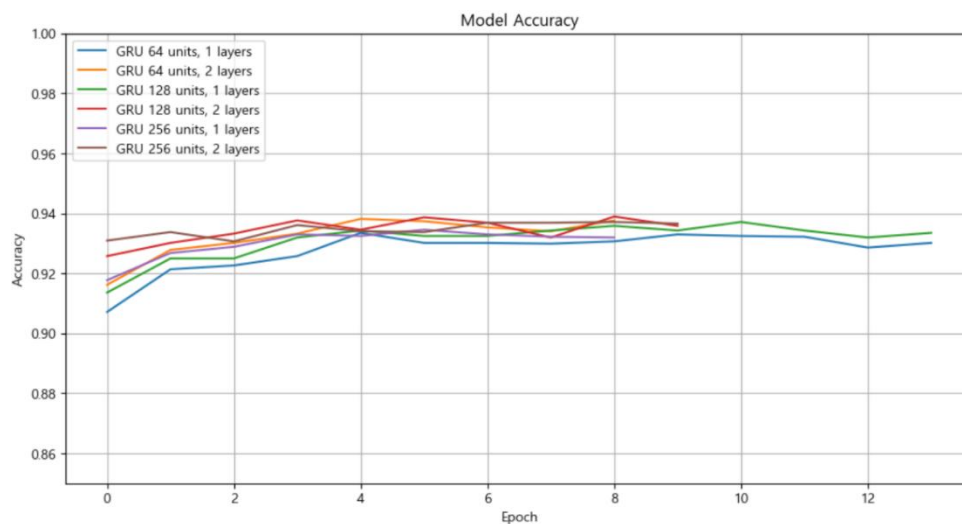
MLP, CNN, LSTM, GRU 4 개의 모델에 대해서 성능비교를 진행하였다. 각 모델별로 은닉층의 수는 2 개로 고정하고 노드의 수를 64, 128, 256 개로 변화를 주며 성능 비교하였고 ACC, AUROC 값 모두 GRU 모델의 성능이 제일 우수하였다. 따라서 본 연구에서 딥러닝 모델 중 최종적으로 GRU 모델을 선정하여 모델 구축하였다.

제 2 절 모델 개선 방안

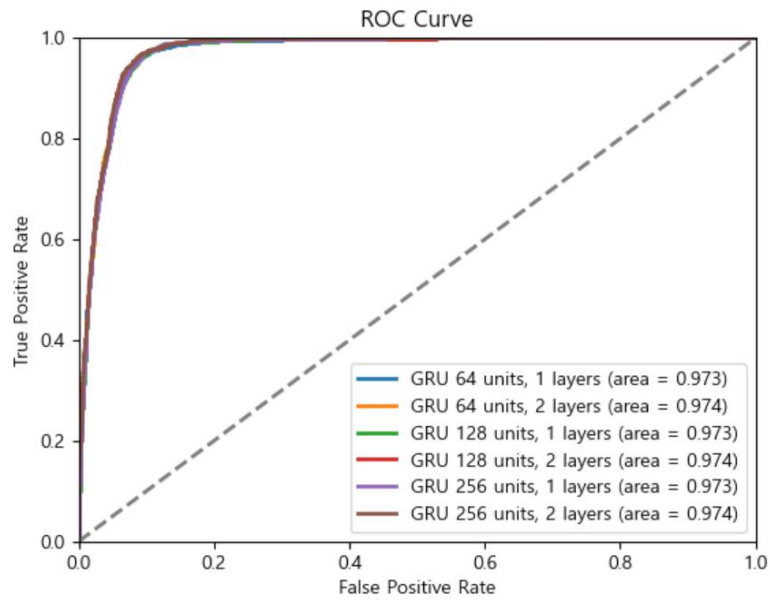
3.2.1. GRU 모델 하이퍼 파라미터

딥러닝 모델로 MLP, CNN, LSTM, GRU 모델 간에 성능 비교를 통해 최종적으로 GRU 모델을 선정하였다. 이후 과정으로 GRU 모델의 하이퍼 파라미터로 은닉층의 수와 노드 수에 변화를 주며 성능을 비교하여 GRU 모델의 하이퍼 파라미터를 결정하고자 하였다.

노드 수의 경우 이전의 모델간 성능 비교할 때와 마찬가지로 64, 128, 256 으로 지정하였고 은닉층의 수를 1, 2 로 지정하여 성능을 비교하였다.

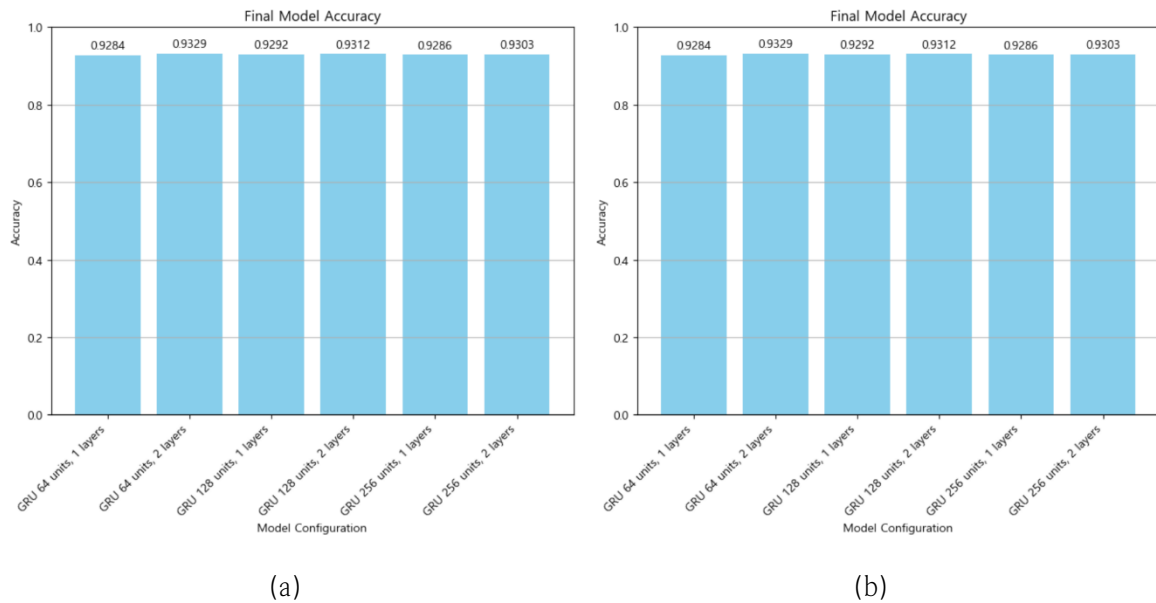


[그림 25. 하이퍼 파라미터 조합별 Accuracy]



[그림 26. 하이퍼 파라미터 조합별 ROC 곡선]

직관적인 이해를 위해 하나의 표 내에서 시각화 하여 성능을 비교하고자 아래와 같이 시각화 하여 Accuracy 와 F1-score 를 비교하였다.



[그림 27. 하이퍼 파라미터 조합별 (a) Accuracy (b) F1-score]

하이퍼 파라미터 조합별 F1-score 값 기준으로 노드 수 64 개, 은닉층 수 2 개일때 성능이 높음을 확인하였다. 따라서 최종적으로 은닉층이 64 개의 노드의 2 개의 층으로 구성된 GRU 모델을 구축하였다.

은닉층이 64 개의 노드의 2 개의 층으로 구성된 GRU 모형의 최적의 epoch 수를 지정하고자 50 번의 epoch 을 지정하여 조기 종료 조건을 걸어주었다. 이를 통해 학습과정에서 발생하는 validation loss 값을 통해 최적의 epoch 을 구하고자 하였다.


```

# 모델 구축
model = Sequential()
model.add(GRU(64, activation='relu', return_sequences=True, input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])))
model.add(GRU(64, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer=Adam(), metrics=['accuracy'])

# 조기 종료 콜백 설정
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# 모델 학습
history = model.fit(X_train_lstm, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1, callbacks=[early_stopping])

# 모델 예측
predictions_proba = model.predict(X_test_lstm)
predictions = (predictions_proba > 0.5).astype(int)

# 모델 평가
accuracy = model.evaluate(X_test_lstm, y_test, verbose=0)[1]
f1 = f1_score(y_test, predictions)

print(f"Accuracy: {accuracy:.4f}, F1 Score: {f1:.4f}")

```

[그림 28. 조기 종료 조건을 걸어 모델 학습]

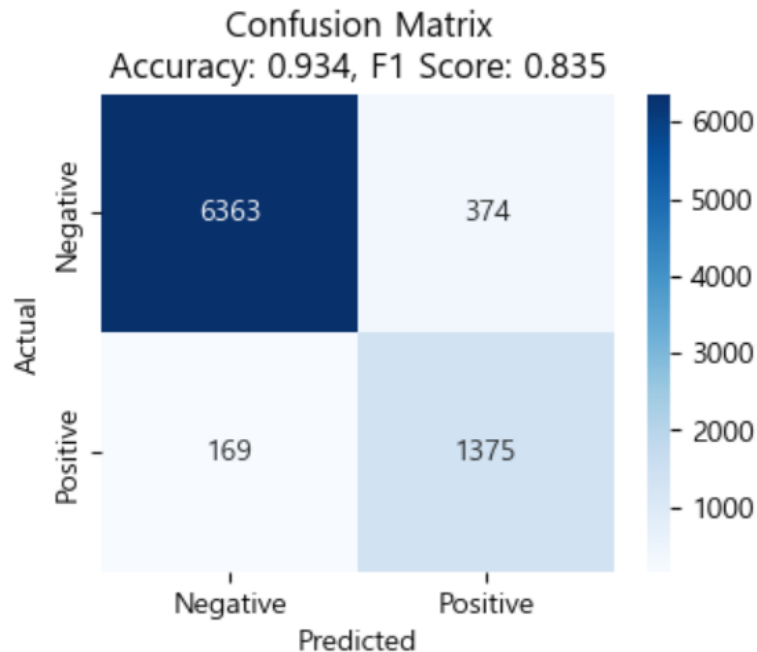
```

Epoch 1/50
484/484 [=====] - 3s 3ms/step - loss: 0.2477 - accuracy: 0.8944 - val_loss: 0.1896 - val_accuracy: 0.921
Epoch 2/50
484/484 [=====] - 1s 2ms/step - loss: 0.1730 - accuracy: 0.9253 - val_loss: 0.1677 - val_accuracy: 0.927
Epoch 3/50
484/484 [=====] - 1s 2ms/step - loss: 0.1580 - accuracy: 0.9335 - val_loss: 0.1637 - val_accuracy: 0.930
Epoch 4/50
484/484 [=====] - 1s 2ms/step - loss: 0.1484 - accuracy: 0.9356 - val_loss: 0.1545 - val_accuracy: 0.934
Epoch 5/50
484/484 [=====] - 1s 2ms/step - loss: 0.1437 - accuracy: 0.9386 - val_loss: 0.1486 - val_accuracy: 0.939
Epoch 6/50
484/484 [=====] - 1s 2ms/step - loss: 0.1396 - accuracy: 0.9390 - val_loss: 0.1485 - val_accuracy: 0.934
Epoch 7/50
484/484 [=====] - 1s 2ms/step - loss: 0.1335 - accuracy: 0.9409 - val_loss: 0.1492 - val_accuracy: 0.934
Epoch 8/50
484/484 [=====] - 1s 2ms/step - loss: 0.1291 - accuracy: 0.9437 - val_loss: 0.1551 - val_accuracy: 0.934
Epoch 9/50
484/484 [=====] - 1s 2ms/step - loss: 0.1271 - accuracy: 0.9444 - val_loss: 0.1478 - val_accuracy: 0.934
Epoch 10/50
484/484 [=====] - 1s 2ms/step - loss: 0.1229 - accuracy: 0.9462 - val_loss: 0.1621 - val_accuracy: 0.932
Epoch 11/50
484/484 [=====] - 1s 2ms/step - loss: 0.1206 - accuracy: 0.9460 - val_loss: 0.1501 - val_accuracy: 0.940
Epoch 12/50
484/484 [=====] - 1s 2ms/step - loss: 0.1155 - accuracy: 0.9488 - val_loss: 0.1652 - val_accuracy: 0.933
259/259 [=====] - 0s 845us/step
Accuracy: 0.9319, F1 Score: 0.8259

```

[그림 29. 조기 종료 조건을 걸어 모델 학습한 결과]

epoch 9 일때 val_loss 값이 0.1478 로 다른 epoch 의 val_loss 값보다 작음을 확인할 수 있다. 따라서 최종 GRU 모델의 epoch 은 9 로 지정하여 학습을 진행하였다. 구축한 모델의 혼동 행렬은 다음과 같다.

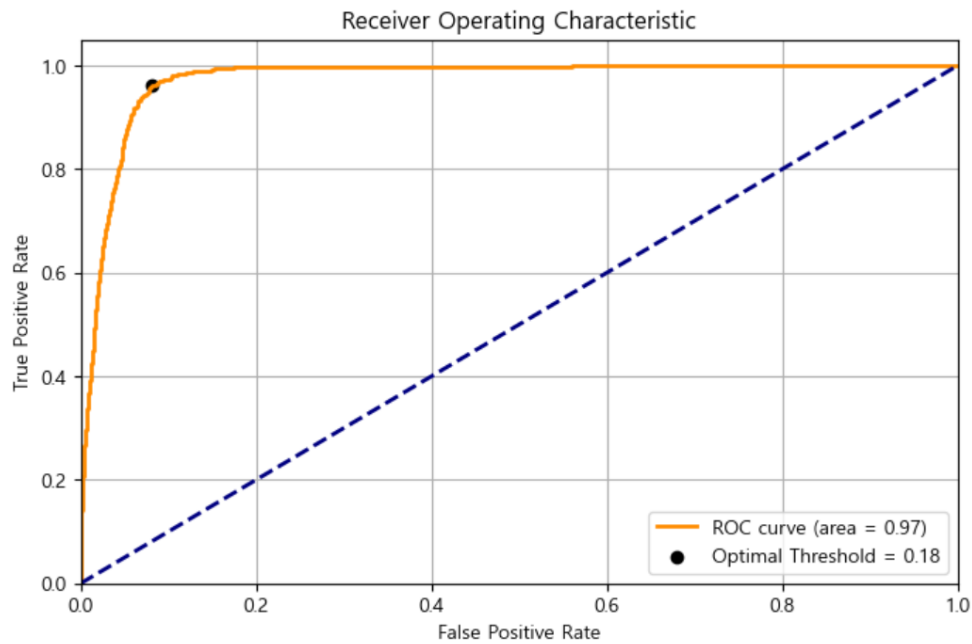


[그림 30. 최종 구축한 GRU 모델의 혼동 행렬]

3.2.2. 최적의 스레스홀드

모델이 예측값을 구해낼 때 중요한 요소 중 하나로 스레스홀드라고 할 수 있다. 모델의 기본 스레스홀드 값인 0.5 를 기준으로 분류해낸다. 본 연구에서는 이진분류인 점과 함께 데이터 불균형인 조건에서 스레스홀드를 단순 0.5 로 지정하여 부실기업 여부를 분류한다면 모델의 성능이 좋기를 기대할 수 없을 것이다.

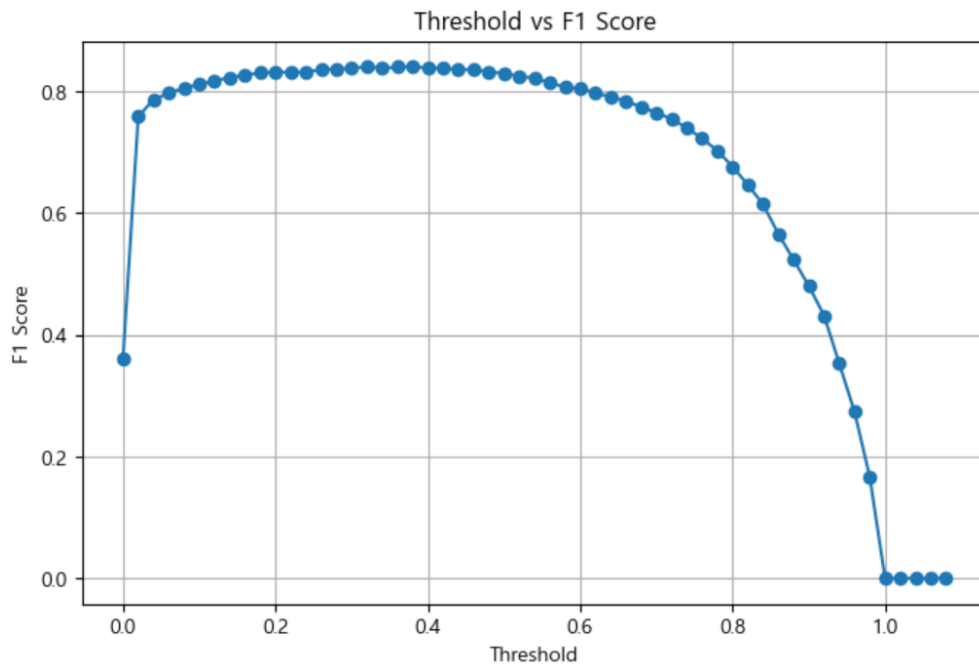
최적의 스레스홀드를 찾고자 Roc curve 를 이용하여 구하였다. 계산식으로 TPR - FPR 를 이용하여 해당 값이 최대가 되는 지점의 스레스홀드를 구하였고 이를 최적의 스레스홀드로 지정하였다.



[그림 31. ROC 곡선을 활용한 최적의 스레스홀드 구하는 과정]

최적의 스레스홀드로 0.18 을 구해내었다. 스레스홀드 0.18 을 기준으로 높은 예측 확률을 뽑아낸다면 해당 기업은 부실 기업으로 예측하고 반대인 0.18 보다 작은 예측 확률을 보일때는 정상 기업으로 분류하도록 하였다.

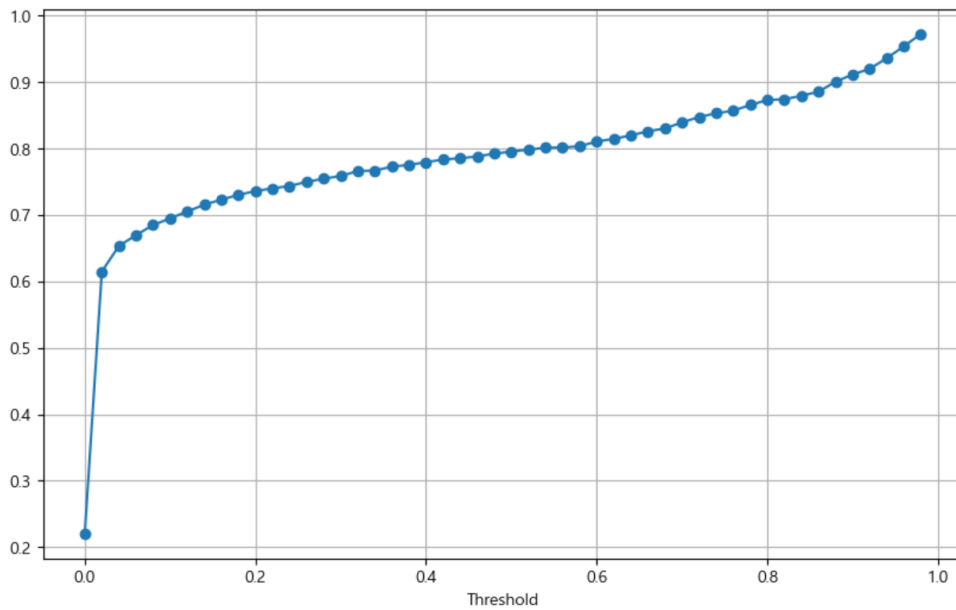
본 연구에서는 최적의 스레스홀드로 0.18 을 선정하였다. 이에 더 나아가 높은 예측 확률을 보이는 기업에 대해서는 더 높은 위험 신호를 주고자 하였다. 이에 스레스홀드 값에 따라 F1-score 를 시각화 하여 어떠한 지점에서 더 높은 위험 신호인 매우 위험의 신호를 줄지에 대한 연구를 하였다.



[그림 32. ROC 곡선을 활용한 최적의 스레스홀드 구하는 과정]

구간별 스레스홀드 값에 따라 F1-score 를 시각화 하였을 때 약 0.1 지점부터 0.6 까지 F1-score 값이 0.8 이상임을 보여주고 있고 0.6 이상의 스레스홀드 값에서는 F1-score 값이 급격히 떨어짐을 확인할 수 있다. 이에 스레스홀드 0.6 지점에서 부실기업으로 예측된 기업 중에서도 특히 더 위험하다는 신호를 주고자 하였다.

구간별 스레스홀드 값에 따라 부실기업으로 예측한 기업 중 실제 부실기업의 비율을 시각적으로 확인하고자 스레스홀드 값에 따라 Precision 값을 시각화 하였다.



[그림 33. 스레스홀드 값에 따른 precision]

구간별 스레스홀드 값에 따라 Precision 값을 시각화 하였을 때 스레스홀드 0.18 인 지점에서 약 0.7 값을 보였고, 스레스홀드 0.6 인 지점에서 약 0.8 값을 보이고 있다. 이에 구축된 모델이 기업을 예측하여 뽑아낸 예측확률에 대해서 0.18 이상인 기업들과 스레스홀드 0.6 이상인 기업들 간에 유의미한 차이가 있음을 확인할 수 있고 이에 최종적으로 예측확률 기준으로 0.18 이상인 기업에 대해서는 부실기업으로 판정하였고 추가적으로 0.6 이상인 기업에 대해서는 매우 위험 단계임을 출력하도록 구조화하였다.

3.2.3. 변수간 군집 형성 및 모델 구축

단순히 내년에 부실기업 여부만을 제공하는 것이 아닌 어떠한 요인에 의해 내년에 해당 기업이 부실기업으로 이어지게 되는지 제공하여 설명력을 강화하고자 하였다. 이를 위해 구축한 최종 데이터셋의 변수들을 한국기업평가⁹에서 활용하고는 5 개의 군집으로 분류하였다.

군집	변수
수익성 비율	총자본영업이익율, 기업법인세비용차감전순이익율, 경영자본영업이익율, 매출액법인세비용차감전순이익율, 자본금법인세비용차감전순이익율, 자본금순이익율, 순이익(종업원1인당), 법인세차감전순이익이자보상비율, 영업이익이자보상비율
레버리지 비율	차입금평균이자율, 금융비용/총비용비율
유동성 비율	유동부채비율, 순운전자본/총자본비율, 현금비율
자본 비율	사내유보율, 유보액/총자산비율, 임차료(구성비)
효율성 비율	EBITDA대금융비용, 법인세차감전순이익(구성비), 조세/조세차감전순이익비율, 금융비용(구성비)

[표 2. 변수별 군집화 한 변수목록]

각 군집별 설명으로 다음과 같다. 수익성 비율은 구체적인 부실기업 판정 기업이 얼마나 효율적으로 수익을 창출하고 있는지를 평가한다. 레버리지 비율은 부채를 통해 자금을 조달하는 정도와 관련된 비율로, 차입금평균이자율 등이 이에 해당한다. 유동성 비율은 단기적인 재무 건정성을 평가한다. 자본 비율은 기업의 자본 구조와 자산 조달의 안정성을 평가한다. 효율성 비율은 자산과 자본을 얼마나 효율적으로 사용하는지를 평가한다.

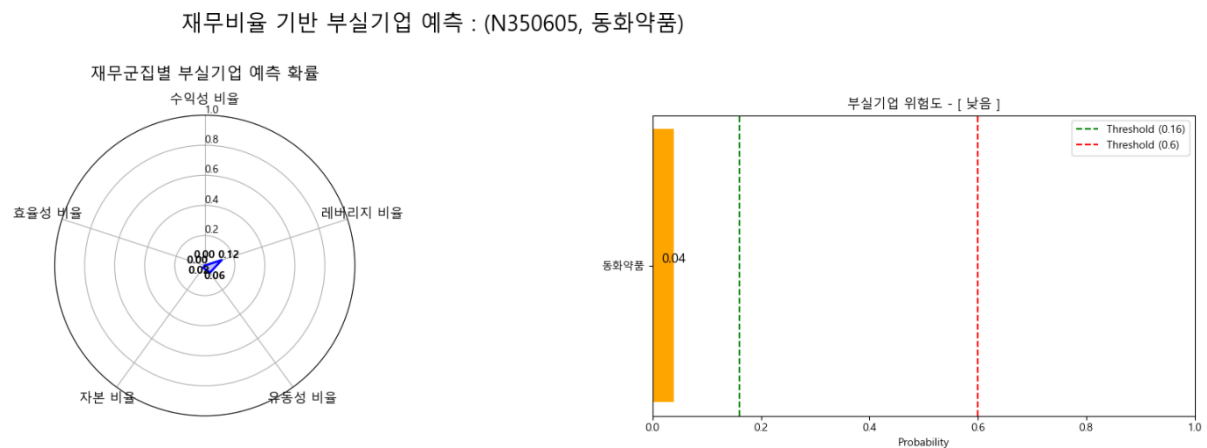
5 가지 영역으로 분류된 변수들에 대해서 각 군집별로 해당하는 변수로 모델을 구축하였다. 5 가지 영역 중 하나인 레버리지 영역에 대해서 모델을 구축하였다면 ‘차입금평균이자율’ 변수와 ‘금융비용/총비용비율’ 변수만을 이용하여 모델을 구축하여 레버리지 요인에 측면에서 부실기업 위험정도를 파악할 수 있고자 하였다.

⁹ <https://www.korearatings.com/index.do>

제 4 장 결론

제 1 절 연구 결과분석

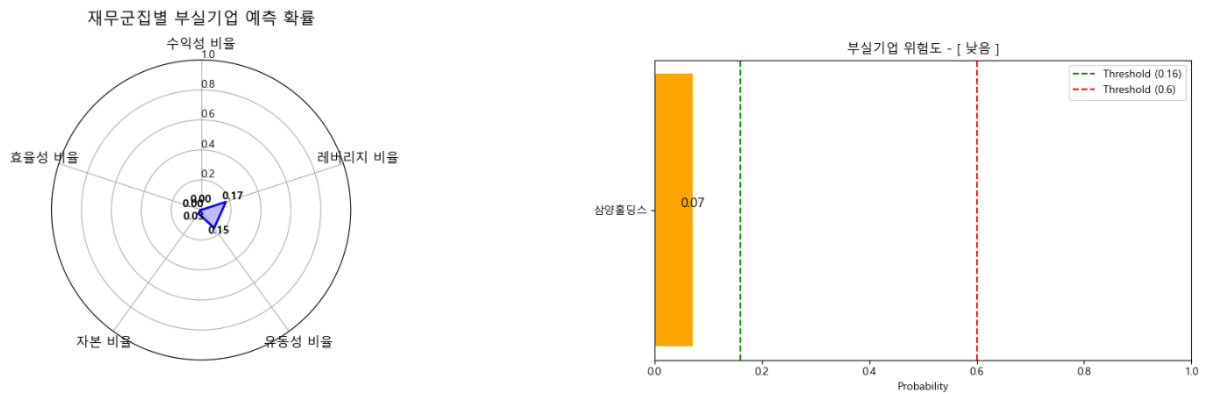
4.1.1. 정상기업



[그림 34. 정상기업의 방사형차트-1]

동화약품은 23 년도에 정상기업으로 분류되었으며, 모형으로 예측한 결과 부실기업 위험도가 0.04 수치로 안전으로 예측되었다. 재무 군집별 부실기업 예측 확률결과는 수익성 비율 0.00, 레버리지 비율 0.12, 유동성 비율 0.06, 자본 비율 0.02, 효율성 비율 0.00 으로 나타났다.

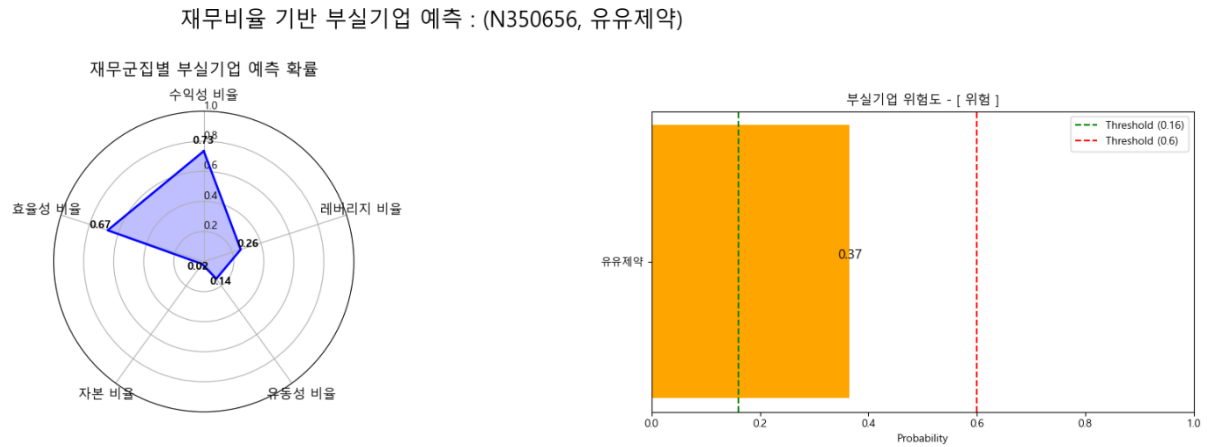
재무비율 기반 부실기업 예측 : (N320730, 삼양홀딩스)



[그림 35. 정상기업의 방사형차트-2]

삼양홀딩스는 23 년도에 정상기업으로 분류되었으며, 모형으로 예측한 결과 부실기업 위험도가 0.07 수치로 안전으로 예측되었다. 재무 균집별 부실기업 예측 확률결과는 수익성 비율 0.00, 레버리지 비율 0.17, 유동성 비율 0.15, 자본 비율 0.03, 효율성 비율 0.00 으로 나타났다.

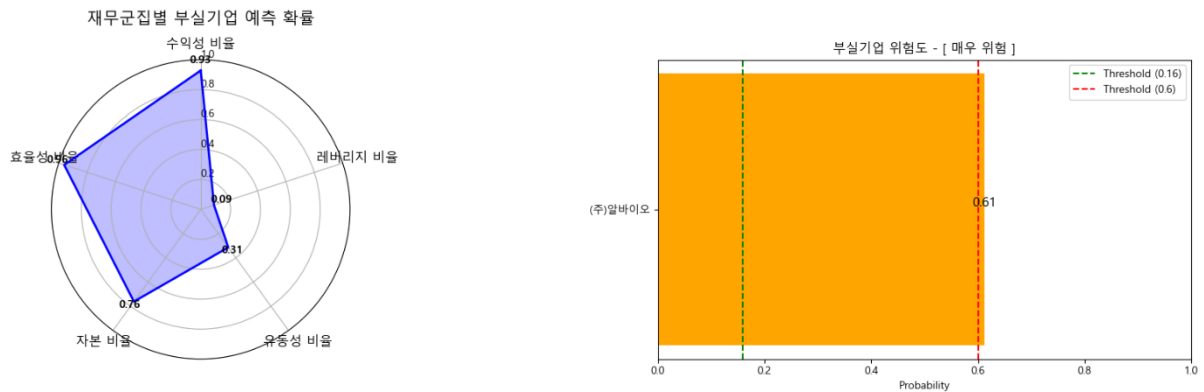
4.1.2. 부실기업



[그림 36. 부실기업의 방사형차트-1]

유유제약은 23 년도에 부실기업으로 분류되었으며, 모형으로 예측한 결과 부실기업 위험도가 0.37 수치로 위험으로 예측되었다. 재무 군집별 부실기업 예측 확률결과는 수익성 비율 0.73, 레버리지 비율 0.26, 유동성 비율 0.14, 자본 비율 0.02, 효율성 비율 0.67 으로 나타났다.

재무비율 기반 부실기업 예측 : (N340286, (주)알바이오)



[그림 37. 부실기업의 방사형차트-2]

(주)알바이오는 23 년도에 부실기업으로 분류되었으며, 모형으로 예측한 결과 부실기업 위험도가 0.61 수치로 매우 위험으로 예측되었다. 재무 균집별 부실기업 예측 확률결과는 수익성 비율 0.93, 레버리지 비율 0.09, 유동성 비율 0.31, 자본 비율 0.76, 효율성 비율 0.96 으로 나타났다.

제 2 절 연구한계점 및 제언

본 연구에는 다음과 같은 한계점이 존재한다.

1. 재무비율 요소만 반영

기업과 관련된 변수 중 대내적 변수인 재무비율만을 이용하여 한계기업을 예측하고자 하였다. 기업의 성과는 기업 그 자체의 운영 성과 이외에도 다양한 대외적인 요인과 환경적 변화에 영향을 받으나, 이러한 기업 외적인 변수는 모형의 입력변수로 활용하지 않았다. 따라서 향후에는 기업의 성과에 영향을 미치는 환경적 요인들을 고찰하고 예측을 위한 변수로 활용할 필요가 있을 것으로 보인다.

2. 최신정보 반영부족

기업의 회계정보만을 활용하여 한계기업을 예측하고자 하였다. 회계정보는 기업의 성과를 나타내는 정보이고 기업의 미래 성과에 대해 예측력을 가진다. 하지만 기업의 주가와 같은 시장정보가 가지는 적시성, 미래지향성 등이 부족하다는 단점이 존재한다. 그러므로 기업과 관련된 이러한 시장정보를 포함하여 분석한다면 모형들의 예측력을 향상시킬 수 있을 것으로 기대된다.

3. 외부감사 대상이 아닌 기업 미반영

마지막으로 본 연구에서 분석에 사용한 데이터는 일정한 기업 규모 이상의 외부감사 대상기업 및 상장기업만으로 구성하였다. 우리나라 경제는 이러한 기업들보다도 중소기업이 그 절대적인 비중을 가지고 있을 뿐만 아니라 경제 상황에 따라 기업부실화로 이어지는 가능성이 중소기업에서 더 높다는 점을 감안할 때, 본 연구는 중소기업을 분석대상에 포함하지 않아 다소 편향된 연구결과를 제시할 가능성을 내포하고 있다. 향후에는 이러한 중소기업만을 대상으로 하거나 혹은 중소기업을 분석대상에 포함하여 연구를 수행할 필요가 있을 것이다.

부실 기업을 조기에 식별하는 것은 매우 중요한 작업이다. 이를 통해 기업의 재정적 위기를 예방하고, 주주 및 이해관계자의 손실을 최소화할 수 있다. 부실 기업의 증가는 경제 전반에 걸쳐 심각한 영향을 미치며, 이를 방지할 경우 금융 시스템과 경제 안정성에 중대한 위협이 될 수 있다. 안정된 금융 환경을 위해서는 이러한 문제들을 조기에 예방할 필요가 있다. 따라서 효과적인 예측 모델의 개발이 필수적이며, 이는 장기적으로 전체 경제 시스템의 안정화에도 중요한 역할을 할 것이다.

참고문헌

Cho, Jae-Hyuk, Ahn, Eun-Ju, & Kim, Sung-Soo. (2021). A Study on the Prediction Model of Insolvent Companies Based on Deep Learning. *Journal of Management Research*, 36(1), 99-113.

Ahn, Hyun-Chul, & Kim, Sung-Jin. (2016). Application of Random Forest for Corporate Credit Rating Prediction. *Journal of Industrial Innovation Research*, 32(1), 187-211.

Lee, Jun-Ki. (2021). A Study on Early Prediction Models of Corporate Bankruptcy: Focusing on Machine Learning Techniques [Master's thesis, Yonsei University]. Retrieved from <http://www-riss-kr.yproxy.hufs.ac.kr/link?id=T15731994>

Park, Joong-Hyun, Kim, Kyung-Young, Joo, Jin-Hyuk, Lee, Hyun, & Choi, Hee-Jung. (2023). Corporate Bankruptcy Prediction Using Deep Learning through Sentiment Analysis Based on ALBERT. *Journal of the Korea IT Policy & Management Society*, 15(3), 3281-3291.

Kim, Yong-Hwan. (2023). A Study on Machine Learning-Based Corporate Credit Rating Models Using Unstructured Data [Doctoral dissertation, Soongsil University]. Retrieved from <http://www-riss-kr.yproxy.hufs.ac.kr/link?id=T16599980>

